CMSC131 (03XX–04XX) Exam 2 Practice

By Naren Ayinala

November 2026

Introduction

Hey everyone!

This is a practice document for your upcoming CMSC131 Exam 2. The primary goal of this document is to help you prepare by reviewing important concepts and practicing the types of problems you may encounter.

However, completing every question in this document does not guarantee that you are fully prepared for the exam. Instead, use this as a preparation tool to help identify topics you understand well and areas where you may need some additional review.

As you work through the problems, try to understand why each solution works rather than simply memorizing answers. If you struggle with a question, use it as a sign that the corresponding topic may be worth reviewing again.

Good luck studying, and I hope this helps you prepare! :)



Short Answer Questions

Question 1. What is the difference between a constructor and a regular method? Can a constructor have a return type?

Question 2. What happens if a class does not define any constructors? What changes when you define a constructor that takes parameters?

Question 3. Can a Java class have multiple constructors? How does Java decide which one to call?

Question 4. What does the `this` keyword mean? Give an example where leaving it out causes a logical error.

Question 5. Distinguish instance variables, static variables, and local variables. When do they receive default values, if ever?

Question 6. Why can't a static method directly access an instance variable without an object reference? Can an instance method access a static variable?

Question 7. If five objects of the same class are created, how many copies of one static variable exist? Explain.

Question 8. Why are instance variables often private? Do getters and setters always prevent privacy leaks?

Question 9. What happens when you print an object whose class does not override `toString()`?

Question 10. Explain `==` versus `equals()` for objects. When could both return `true`?

Question 11. When writing an `equals()` method, why should you compare relevant field values rather than only object references?

Question 12. How does an array of primitive values differ from an array of object references?

Question 13. Consider the following code:

```
int[] a = new int[5];
String[] b = new String[5];
```

What are the initial element values? Have any `String` objects been created?

Question 14. Compare these two examples:

```
int[] a = {1, 2, 3};
int[] b = a;
```

```
int[] a = {1, 2, 3};
int[] b = new int[a.length];
for (int i = 0; i < a.length; i++) {
    b[i] = a[i];
}
```

What happens to `a` when `b[0]` is modified in each example?

Question 15. Can two variables refer to the same object? What happens if that object is modified through one variable?

Question 16. Explain reference copying, shallow copying, and deep copying. How do they differ when an object contains references to mutable objects?

Question 17. Suppose an array contains references to mutable objects. If you create a new array and copy each reference into it, is that a deep copy? Explain.

Question 18. What is a privacy leak? How could a getter that returns a private array cause one?

Question 19. Can a constructor cause a privacy leak even if its instance variables are private? Explain.

Question 20. Consider:

```
final int[] nums = {10, 20, 30};
```

Can `nums[0]` change? Can `nums` be assigned a different array?

Question 21. What is the difference between a reference variable that is `null` and an array that contains `null` elements?

Question 22. If an array is passed to a method, can the method modify its elements? Can assigning a new array to the parameter change the caller's variable?

Question 23. What does `==` compare when used with two array variables? How could you compare the corresponding elements of two integer arrays without library methods?

Question 24. How are `String` and `StringBuffer` different in terms of mutability?

Question 25. Consider:

```
StringBuffer sb = new StringBuffer("Java");
StringBuffer other = sb;
other.append(" Programming");
```

Does `sb` change? Why?

Question 26. Explain `break` and `continue`. What happens immediately after `continue` in a `for` loop versus a `while` loop?

Question 27. What happens if a `break` is omitted in a traditional `switch` statement? Does `default` always run?

Question 28. What is the ternary operator? Can every `if-else` statement be replaced with one?

Question 29. Distinguish a compilation error from a runtime exception, with an example of each.

Question 30. What is the purpose of `try-catch`? If an exception occurs halfway through a `try` block and is caught, do the remaining statements in the `try` block execute?

Question 31. Give an operation that could cause each of these exceptions: `NullPointerException`, `ArrayIndexOutOfBoundsException`, and `StringIndexOutOfBoundsException`.

Question 32. Does returning an object reference from a method necessarily create a new object? Explain.

Question 33. What is an enhanced `for` loop? Give one situation in which an indexed `for` loop is more appropriate.

Code Tracing and Debugging

For each question, determine the exact output or identify any compilation or runtime error. Explain your reasoning. Each question is independent. Assume code fragments appear inside a valid method unless a complete class is shown.

Question 1. What is the exact output?

```
int a = 7;
int b = 4;
int result = (a % 2 == 0) ? a + b : a - b;
System.out.println(result);
System.out.println((result > 5) ? "High" : "Low");
```

Question 2. What is the exact output? Pay attention to side effects.

```
int x = 3;
int y = (x++ >= 3) ? x + 2 : --x;
System.out.println(x);
System.out.println(y);
```

Question 3. What is the exact output of this nested ternary expression?

```
int score = 76;
String level = (score >= 90) ? "A" :
               (score >= 75) ? "B" :
               (score >= 60) ? "C" : "D";
System.out.println(level);
System.out.println((score % 2 == 0) ? 1 : 0);
```

Question 4. What is the exact output? Explain the role of `break`.

```
int day = 2;
switch (day) {
    case 1:
        System.out.print("Mon ");
    case 2:
        System.out.print("Tue ");
    case 3:
        System.out.print("Wed ");
        break;
    default:
        System.out.print("Other ");
}
System.out.println("Done");
```

Question 5. What is the exact output?

```
char code = 'B';
int value = 1;
switch (code) {
    case 'A':
        value += 2;
        break;
```

```
    case 'B':
        value *= 3;
    case 'C':
        value += 4;
        break;
    default:
        value = -1;
}
System.out.println(value);
```

Question 6. What is the exact output?

```
int[] nums = {3, 6, 9, 12};
for (int i = 0; i < nums.length; i++) {
    if (i % 2 == 0) {
        nums[i] += nums[1];
    }
}
for (int n : nums) {
    System.out.print(n + " ");
}
```

Question 7. What is the exact output?

```
int[] first = {2, 4, 6};
int[] second = first;
second[1] = 10;
first = new int[]{1, 3, 5};
System.out.println(second[0] + second[1]);
System.out.println(first[1]);
```

Question 8. Does this code compile? If so, what happens when it runs?

```
String[] names = new String[3];
names[0] = "Ada";
names[2] = "Grace";
for (int i = 0; i < names.length; i++) {
    System.out.println(names[i].length());
}
```

Question 9. Consider the following class and statements. What is printed?

```
public class Ticket {
    private int number;
    private static int created = 0;

    public Ticket(int number) {
        this.number = number;
        created++;
    }

    public void change(int n) {
        number += n;
    }
```

```
    public String toString() {
        return number + ":" + created;
    }
}

Ticket a = new Ticket(5);
Ticket b = new Ticket(8);
a.change(2);
System.out.println(a);
System.out.println(b);
```

Question 10. Consider the following class. What is printed? Explain the difference between the constructor parameter and the instance variable.

```
public class Lamp {
    private int brightness;

    public Lamp(int brightness) {
        brightness = brightness + 1;
    }

    public void setBrightness(int brightness) {
        this.brightness = brightness;
    }

    public String toString() {
        return "Brightness: " + brightness;
    }
}

Lamp a = new Lamp(4);
System.out.println(a);
a.setBrightness(7);
System.out.println(a);
```

Array Coding Questions

Write complete static Java methods. Unless otherwise specified, assume input arrays are non-null. Do not use `ArrayList`, streams, sorting methods, or other collections.

Question 1. Write a method with the following signature:

```
public static int countMostFrequent(int[] arr)
```

Return the number of occurrences of the most frequently occurring integer. For example, `{4, 2, 4, 3, 2, 4, 5}` gives `3`. Return `0` for an array. Do not modify the array. You may use nested loops.

Question 2. Write a method with the following signature:

```
public static int countAboveAverage(int[] arr)
```

Return the number of elements strictly greater than the arithmetic average of all elements. For example, `{2, 4, 6, 8}` gives `2`. Return `0` for an empty array. Do not modify the original array. Be careful about integer division.

Question 3. Write a method with the following signature:

```
public static int[] reverseArray(int[] arr)
```

Return a *new* array with the elements in reverse order. For example, `{3, 7, 1}` produces `{1, 7, 3}`. Do not change the original array. An empty array should produce a new empty array.

Question 4. Write a method with the following signature:

```
public static int[] removeTarget(int[] arr, int target)
```

Return a new array containing all values except those equal to `target`, in their original order. For example, `removeTarget(new int[]{2, 5, 2, 7, 2}, 2)` produces `{5, 7}`. The result must have exactly the necessary length. Do not modify the original array.

Question 5. Write a method with the following signature:

```
public static int secondLargest(int[] arr)
```

Return the second-largest *distinct* integer in the array. For example, `{8, 3, 8, 5}` gives `5`. You may assume the array contains at least two distinct values. Do not sort or modify the array.

Class Writing Practice

Write complete Java classes, including the specified fields, constructors, and methods. Follow the requirements carefully. For array fields, use loops to make deep copies where necessary; do not use `clone()`.

Question 1. Implement a class named `BankAccount`.

- Private instance variables `String owner` and `double balance`, and a private static `int accountCount` initially `0`.
- A constructor receiving owner and initial balance. Treat a negative initial balance as `0`. Increment `accountCount` once per constructed object.
- A `deposit(double amount)` method that deposits only positive amounts.
- A `withdraw(double amount)` method returning `true` only for a positive amount no greater than the current balance; subtract it when successful.
- Getters for owner and balance, and a static `getAccountCount()` method.
- A `toString()` returning text like `Alex: $150.0`.
- An `equals(BankAccount other)` method comparing owner and balance; handle a `null` argument. Use `String.equals()` for owner.

Question 2. Implement a class named `Course`.

- Private fields `String courseName` and `int[] grades`.
- A constructor receiving a non-null name and non-null grades array; make a deep copy of the grades array.
- A copy constructor that makes a deep copy of another `Course`.
- `getGrades()` returning a deep copy of the grades array.
- `setGrade(int index, int grade)` changing a value only for a valid index and a grade from `0` through `100`.
- `average()` returning the mean as a `double`, or `0.0` for an empty array.
- `toString()` displaying the course name and grades in order.
- `equals(Course other)` comparing the course name and all grades, and handling `null`. Do not use array comparison library methods.

Question 3. Implement a class named `Movie`.

- Private fields `String title`, `int year`, `double rating`, and private static `int totalMovies`.

- One constructor receiving all three instance values, and another receiving only title and year (rating defaults to `0.0`). Avoid duplicating initialization code. Increment the count exactly once per object.
- Getters for all instance variables, and a rating setter accepting only values from `0.0` to `10.0` inclusive.
- Static `getTotalMovies()`.
- `toString()` producing text like `Inception (2010):  8.8`.
- `equals(Movie other)` comparing title and year, ignoring rating, and handling `null`.

Question 4. Implement a class named `TemperatureTracker`.

- Private fields `String location` and `double[] temperatures`.
- A constructor receiving a non-null location and non-null array; make a deep copy of the array.
- A copy constructor that makes a deep copy of another `TemperatureTracker`.
- `getTemperatures()` returning a deep copy of the array.
- `setTemperature(int index, double value)` updating a valid index only.
- `getAverage()` returning the average, or `0.0` if empty.
- `getMaximum()` returning the maximum, or `0.0` if empty.
- `countAbove(double threshold)` returning how many readings exceed the threshold.
- `toString()` displaying the location and all recorded temperatures.

Question 5. Implement a class named `StudentRecord`.

- Private fields `String name`, `int id`, and `int[] scores`.
- A constructor receiving non-null name and scores array; make a deep copy of the scores array.
- A copy constructor making a deep copy of another `StudentRecord`.
- Getters for name, ID, and scores; the scores getter must return a deep copy.
- `setScore(int index, int score)` updating a valid index only if the score is between `0` and `100` inclusive.
- `getHighestScore()` returning the maximum, or `-1` if empty.
- `getAverage()` returning the average as a `double`, or `0.0` if empty.
- `toString()` displaying name, ID, and scores.
- `equals(StudentRecord other)` comparing ID and every score, ignoring name; handle `null`. Do not use array comparison library methods.

A Final Note :)

Even though you will not be writing pseudocode on the exam, practice thinking through your solution before coding. If you get stuck on a question during the exam, do not panic or spend all your time staring at it. Move on and come back to it later. Sometimes coming back with a clear mind is all you need. You might even see something in another question that reminds you of an idea or gives you a small hint about how to approach the question you skipped.

When practicing, if your code is not working, try not to immediately use AI or look up the solution. Trace your own code first. Go through it iteration by iteration and keep track of how your variables are changing. Ask yourself what each line is doing and compare what you expected to happen with what is actually happening. Learning how to debug your own code will help you much more on the exam than simply seeing the correct solution (come to Office Hours if you feel stuck anywhere!!).

I also highly recommend looking through your projects again before the exam. Do not just look at whether you got them correct, rather look at the code you wrote and make sure you understand why it works. Your projects contain a lot of the concepts you have been practicing throughout the semester.

Exam logistics and past exams (I highly recommended past papers for practice):
`https://www.cs.umd.edu/class/fall2026/cmsc131-03XX-04XX/exams/exam2/`

